Decoupling Memory from Context: Structured Memory for Token-Efficient Test-Time Continual Learning

Yehya Farhat
Rice University, Department of Computer Science
Houston, TX, USA
Yehya.Farhat@rice.edu

Michael Desmond
IBM Research
Yorktown Heights, NY, USA
mdesmond@us.ibm.com

Anastasios Kyrillidis
Rice University, Department of Computer Science
Houston, TX, USA
anastasios@rice.edu

Abstract

Large language models (LLMs) are increasingly deployed in enterprise, scientific, and medical applications, where agents must incorporate domain-specific knowledge and adapt from experience. Context engineering offers a practical alternative to weight updates by improving model behavior through instructions, strategies, and evidence supplied at inference time. However, adapting context online typically requires a costly trial-and-error process, while queries are often processed independently, preventing useful experience from carrying forward. Memory systems address this limitation by retaining information across interactions, but approaches that continually append information to a shared context face increasing token costs, context-window limits, and performance degradation as the context expands. We introduce a unified formulation of context optimization and show that an agent memory system update can be interpreted as optimization update procedure over the model's context. This perspective attempts to provide a principled framework for studying memory design and its efficiency. We then propose GraphMemory, a lightweight graph-based memory that accumulates, refines, organizes, and connects reusable strategies. For each query, GraphMemory retrieves only the relevant subgraph, enabling online context adaptation without exposing the model to the entire memory. Under bounded retrieval, the amount of retrieved memory remains constant as the number of processed examples grows. Experiments show that GraphMemory achieves competitive downstream performance while using approximately 83–95% fewer memory-construction tokens than our baselines.

1 Introduction

Modern large language models (LLMs) achieve strong performance across reasoning, question answering, and open-ended generation tasks [20]. Yet deployed agents largely remain static: with fixed parameters, information acquired from one interaction does not naturally improve later behavior. Parameter-efficient fine-tuning (PEFT) provides one mechanism for adaptation [8, 6, 11], while more recent methods generate context-specific adapters on the fly in a single forward pass [4]. These approaches still require access to model weights and training infrastructure, and encode their learned state in model-specific parameters. This limits continual adaptation during deployment and makes the acquired knowledge difficult to transfer across agents.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Towards Test-Time Continual Learning Agents.

Context engineering offers a weight-free alternative by adapting a frozen model through instructions, demonstrations, strategies, or evidence supplied alongside each query to leverage its in-context learning abilities [3]. Prompt-optimization methods treat this context as the optimization variable and search for task-level prompts that improve performance [23, 10, 14, 1]. However, they generally optimize a single prompt against an offline dataset or user-defined evaluator rather than accumulating experience during deployment. Other methods use a hybrid setup that derives training signals from model behavior itself [16]. Persistent memory systems instead update an external natural-language memory as new problems are solved. Dynamic Cheatsheet stores reusable strategies in a persistent file [17], while Agentic Context Engineering (ACE) incrementally evolves a structured playbook through a Generator–Reflector–Curator architecture [25]. Because this memory remains external to the model, it can be inspected, edited, and transferred across agents and has shown success in large-scale enterprise applications [18, 13].

Nevertheless, scaling such memory introduces a fundamental problem. Because existing systems largely maintain a flat collection or a single evolving file, the model must process an increasingly large and heterogeneous memory context for every query. This increases token usage, pushes against context-window limits, and can lead to context degradation or collapse. To address these limitations, we introduce GRAPHMEMORY, a structured external memory for scalable test-time learning. GRAPHMEMORY represents reusable ACE-style knowledge points as nodes and captures useful relationships between them as weighted edges. For each query, the system identifies a small set of relevant entry nodes and traverses their local neighborhoods, exposing only the resulting subgraph to the language model. This representation enables selective retrieval without requiring all experience to be compressed into one global playbook or assigned exclusively to an isolated section. It also makes relationships among individual lessons explicit, allowing knowledge acquired from different experiences to be retrieved together when their connection is useful to the current problem. Our contributions are listed as follows:

- **A unified context-optimization framework.** We formulate adaptation of a frozen language model as optimization over its textual context and show that persistent memory systems can be interpreted as optimization procedures that update this context across examples. Under bounded update sizes, we show that the context used by append-only memory grows as $O(N)$, whereas bounded retrieval maintains an $O(1)$ memory-context length per query.
- **GraphMemory.** We introduce GRAPHMEMORY, a lightweight external memory that accumulates, refines, organizes, and connects reusable strategies in an evolving graph. Nodes represent ACE-style knowledge points, while weighted edges capture useful relationships among them. For each query, GraphMemory identifies relevant entry nodes, traverses their local neighborhoods, and supplies only the resulting subgraph to the language model. This replaces full-memory prompting with selective relational retrieval while preserving the interpretability, portability, and weight-free adaptation of natural-language memory.
- **Competitive performance at substantially lower cost.** Across four dataset–model settings on two financial reasoning tasks, FORMULA and FINER, LLM-routed GraphMemory remains within 4.0 percentage points of ACE with Claude Haiku 4.5 and outperforms ACE by 1.0 points on FORMULA with Qwen3.8-27B. The retrieval variants reduce training-token usage by 83–95%, test-token usage by 79–98%, and active training time by 21–34% in the Claude runs.

2 Context and Memory Formalization

2.1 Offline Context Optimization

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ be an offline training set, let f be a frozen language model, and let $c \in \mathcal{C}$ denote textual context supplied alongside an input. We define the empirical context risk as

$$L_{\mathcal{D}}(c) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}\left[\ell\big(f(c, x_i), y_i\big)\right], \tag{1}$$

where the expectation is over any randomness in the model or its execution environment. Context optimization seeks $c^* \in \arg\min_{c \in \mathcal{C}} L_{\mathcal{D}}(c)$. The context may contain instructions, demonstrations, knowledge, strategies, or any combination thereof. Because c is a discrete sequence of tokens and the model may be accessible only through a black-box interface, ordinary first-order optimization with respect to c is unavailable.

Prompt-optimization methods such as MIPROv2 and GEPA [14, 1] approximate this problem by restricting the search to proposed candidates. Let $\mathcal{P}_T \subset \mathcal{C}$ denote the candidate pool considered within budget T. Their shared abstraction is

$$\widehat{c}_T \in \arg\min_{c \in \mathcal{P}_T} L_{\mathcal{D}}(c). \tag{2}$$

The methods differ in how $\mathcal{P}_T$ is constructed and searched. When abstracted to a single context variable, Equation (2) produces a task-level context that is held fixed on the test set. We give the more specific formulations in Appendix A.1.

A complementary family of methods uses model feedback to iteratively propose textual changes to the current context. Let $\widehat{y}_t = f(c_t, x_t)$, and let τ_t contain the available feedback and execution trace. We abstract such an update as

$$\Delta c_t = \mathcal{O}(c_t, x_t, \widehat{y}_t, \tau_t), \qquad c_{t+1} = c_t \odot \Delta c_t, \tag{3}$$

where $\mathcal{O}$ is a black-box optimization oracle and $\odot$ applies the proposed addition, deletion, replacement, or edit. Equation (3) is structurally analogous to a gradient-descent step over the context: Δc_t plays the role of an update direction. It is not a literal gradient, however, and it need not decrease Equation (1). Methods such as OPRO, TextGrad, and TRACE [5, 24, 22] differ primarily in how they construct the oracle $\mathcal{O}$ and the update. We discuss these realizations in Appendix A.1.

2.2 From Local Context Updates to Memory

Directly asking an imperfect oracle to improve one global context over the entire dataset is difficult. Memory systems instead extract a short context block Δ_i from an individual experience (x_i, y_i). Writing the collection of context blocks as $\delta = (\Delta_1, \ldots, \Delta_n)$, we define the context-block risk as the local surrogate

$$L_{\mathcal{D}}^{\text{block}}(\delta) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}\left[\ell\big(f(\Delta_i, x_i), y_i\big)\right]. \tag{4}$$

This surrogate decomposes update generation into smaller problems: the oracle need only distill a focused lesson, for example, a strategy, correction, or failure mode, from one experience at a time. The blocks collectively form an external memory bank δ, which serves as the evolving optimization state of the memory system.

Optimizing Equation (4) is not claimed to be equivalent to optimizing the global objective in Equation (1). A block may overfit its source example and need not generalize in isolation. The motivation for this surrogate is primarily practical. First, it models the update procedure of a memory system. Second, it makes the oracle's task more local: proposing a focused update Δ_i is a simpler generation task than proposing changes to improve performance over the entire dataset. A separate composition or retrieval mechanism must determine which blocks should form the context for a new input; the local surrogate alone does not ensure generalization. We expand on this distinction in Appendix A.2. Dynamic Cheatsheet and ACE instantiate this view by integrating locally extracted lessons into a shared natural-language memory [17, 25]. The blocks are integrated into one shared context, making accumulated lessons available across examples and producing a single task-level memory for deployment. The resulting memory must therefore serve new inputs rather than supply example-indexed solutions.

Under an append-only abstraction, this context evolves as

$$c_\delta^{(t+1)} = c_\delta^{(t)} \oplus \Delta^{(t)}, \tag{5}$$

where $\oplus$ appends the new block to the shared context c_δ. This is an idealized regime: Dynamic Cheatsheet and ACE may edit, merge, or delete existing content rather than literally appending every update. Nevertheless, these operations do not preclude systematic context growth. In Figure 1, we show that the measured growth of ACE's prompt-token usage is consistent with the growth trend predicted by the update regime in Equation (5) over the evaluated training horizon.

Proposition 1 (Append-only memory-context growth)**.** *Suppose that N examples produce at most one block each and that $\text{len}(\Delta^{(t)}) \leq B$, where B is independent of N. Under Equation* (5)*, with a fixed initial context, the final memory-context length satisfies*

$$\text{len}\left(c_\delta^{(N)}\right) \leq \text{len}\left(c_\delta^{(0)}\right) + NB = O(N). \tag{6}$$

The proof follows by unrolling the induced length recurrence and is given in Appendix A.3.

This growth matters because the shared memory is not stored and read only once: it is repeatedly supplied to the model on subsequent examples. The amount of memory processed per query therefore increases with the number of accumulated time-steps (lessons). Across N sequential updates, this can produce quadratic cumulative memory-context exposure, even though the final context itself grows only linearly. In practice, the expanding context also moves the system toward the model's context-window limit and mixes an increasingly heterogeneous collection of lessons, only a fraction of which may be relevant to the current input. The corresponding cumulative bound is derived in Appendix A.3. The resulting efficiency problem is not that the stored memory grows; a continual learner should retain experience. The problem is that full-memory systems couple stored-memory size to the amount of memory processed on every query. This motivates a memory interface that allows the stored knowledge to continue growing while keeping the active context small.

2.3 Memory with Bounded Retrieval

A more token-efficient procedure introduces an interface that separates the accumulated memory δ from the context c exposed to the model:

$$\delta^{(t+1)} = \delta^{(t)} \cup \{\Delta^{(t)}\}, \qquad c_\phi^{(t+1)}(x) = \phi\big(\delta^{(t+1)}, x\big), \tag{7}$$

where ϕ retrieves a query-dependent subset of the accumulated blocks. The oracle $\mathcal{O}$ determines what is learned from each experience, while ϕ determines which learned information is exposed for a particular input.

To this end, we introduce GraphMemory and model δ as a graph $G^{(t)} = (V^{(t)}, E^{(t)})$. Each node stores a reusable context block Δ, and weighted edges identify blocks that may be useful together. The graph is a modeling choice for instantiating ϕ that supports selective retrieval and composition of related blocks. We test three implementations of ϕ: topk embedding retrieval over the full memory without traversal, LLM retrieval with graph traversal, and embedding retrieval with graph traversal. The graph variants select entry nodes and expand through their neighborhoods; topk directly retrieves the most similar blocks. All use fixed retrieval budgets and supply only the selected blocks to the model f. The complete graph update and its relation to full-memory exposure are given in Appendix A.4; the concrete retrieval and update procedures are described in the following section.

Proposition 2 (Bounded context under retrieval)**.** *Suppose each stored block contains at most B tokens and ϕ retrieves at most K blocks, where B and K are independent of N. Then, for every query x, the retrieved memory context satisfies*

$$\operatorname{len}\left(c_\phi^{(N)}(x)\right) \le KB = O(1) \tag{8}$$

The proof and cumulative memory-context bound are given in Appendix A.3. Proposition 2 concerns only the memory supplied to the Generator; it does not bound storage or the internal cost of retrieval. Figure 1 illustrates the different prompt-token growth trends of ACE and GraphMemory over the evaluated training horizon.

3 GraphMemory

We use ACE as our workhorse for realizing GraphMemory. GraphMemory changes ACE at precisely two interfaces: what the Generator (f) reads and what the Curator ($\mathcal{O}$) writes. Instead of exposing the complete playbook to every query, we prepend a retrieval stage; instead of writing only bullet-point updates, the Curator can also write relations between them. The resulting adaptation loop is RETRIEVE–GENERATE–REFLECT–CURATE, as illustrated in Figure 2. For retrieval, we maintain a compact index containing each node's identifier and a short key, but not its full bullet-point content. Given a query, an LLM or embedding similarity selects a small set of entry nodes. In the graph variants, we then perform a deterministic breadth-first traversal from those entries, expanding neighbors in order of learned edge utility, under fixed hop, node, and token budgets. Only the full contents of the nodes reached during this traversal are loaded and passed to the Generator. The topk variant instead ranks all stored nodes by embedding similarity and loads the best matches directly, without graph traversal.

Generation follows ACE. In the graph variants, reflection is extended in one respect: beyond ACE-style critique, the Reflector labels the Generator's reported memory transitions as helpful, harmful, or

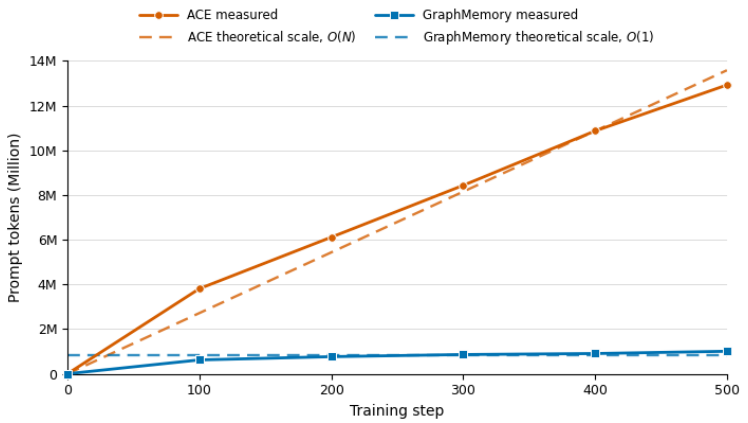


Figure 1: Prompt-token usage at successive training checkpoints for GraphMemory and ACE on FORMULA with Qwen3.8-27B.

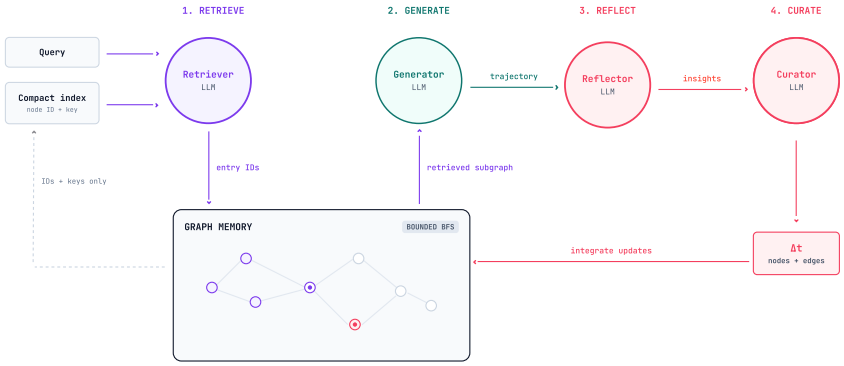


Figure 2: Overview of GraphMemory. An LLM retriever selects entry nodes from a compact index, after which a bounded breadth-first traversal loads the relevant subgraph. The ACE Generator and Reflector operate on this retrieved context, while the Curator writes new nodes and edges back to the graph.

neutral. Helpful and harmful labels contribute confidence-weighted evidence to the corresponding edges, while neutral labels leave the counters unchanged. The traversal utility is the mean of the resulting Beta posterior, so retrieval preferentially follows transitions with stronger helpful evidence. The difference appears during curation: the Curator produces a graph update that may add or revise bullet-point nodes, add or revise edges, or do both. The update is applied locally without rewriting the rest of the graph. New nodes are added to the compact index, while new edges determine which lessons can be reached together during later retrieval. We provide the detailed construction of GraphMemory in Appendix B. It is important to note that the LLM-retrieval variant is not strictly $O(1)$ end to end because its Retriever still reads an index that grows with the number of nodes. The bound in Proposition 2 applies to the full memory content supplied to the Generator. Since each index entry contains only an identifier and a short key, however, this residual growth is far smaller than repeatedly supplying every bullet in full. Over the training horizon studied here, the resulting context grows slowly enough to exhibit the near-constant behavior shown in Figure 1.

4 Experiments

Baselines We compare topk, LLM retrieval with graph traversal, and embedding retrieval with graph traversal against ACE [25] using two state-of-the-art LLM backbones across two financial reasoning tasks. For each backbone and task, we report cold-start performance with an empty memory and final test performance after memory construction. To assess adaptation efficiency, we track cumulative

Table 1: Offline adaptation performance, token usage, and estimated training cost on FORMULA and FINER. Token counts and costs are approximate. Costs assume a $95\% - 5\%$ input–output token split and the fixed prices below. Initial accuracy denotes cold-start test accuracy; final test uses the best validation checkpoint. FiNER accuracy is micro-averaged over tags. Bold indicates the best validation/test accuracy and lowest token/cost totals among methods in each dataset–backbone group.

Dataset	Backbone	System	Accuracy			Token Usage		Estimated Cost*
			Initial	Best val.	Final test	Training	Test	Training (USD)
FORMULA	Claude Haiku 4.5	GraphMemory (LLM)	0.70	0.87	0.81	∼14M	∼1M	$16.80
		GraphMemory (embedding + graph)	0.72	0.87	0.83	∼10M	∼**0.53M**	$12.47
		topk	0.72	0.88	0.85	∼**9M**	∼0.57M	**$11.07**
		ACE (80k cap)	0.70	**0.89**	**0.85**	∼131M	∼18M	$157.26
	Qwen3.8-27B	GraphMemory (LLM)	0.67	0.87	**0.85**	∼7M	∼0.74M	$4.24
		GraphMemory (embedding + graph)	0.65	0.87	0.82	∼7M	∼0.27M	$4.15
		topk	0.66	0.86	0.83	∼**5M**	∼**0.27M**	**$3.33**
		ACE	0.66	**0.89**	0.84	∼41M	∼6M	$25.44
FINER	Claude Haiku 4.5	GraphMemory (LLM)	0.75	**0.81**	0.76	∼57M	∼9M	$68.23
		GraphMemory (embedding + graph)	0.75	0.79	0.74	∼34M	∼3M	$40.56
		topk	0.75	0.79	0.75	∼**29M**	∼**2M**	**$35.34**
		ACE (80k cap)	0.75	0.80	**0.79**	∼327M	∼42M	$392.85
	Qwen3.8-27B	GraphMemory (LLM)	0.73	0.82	0.75	∼31M	∼5M	$19.24
		GraphMemory (embedding + graph)	0.73	0.78	0.73	∼21M	∼**2M**	$12.90
		topk	0.74	0.78	0.77	∼**20M**	∼2M	**$12.41**
		ACE	0.72	**0.83**	**0.80**	∼429M	∼97M	$267.86

*Assumed input/output prices per million tokens: Claude Haiku 4.5, $1.00/$5.00; Qwen3.8-27B, $0.50/$3.00. Qwen costs are API-equivalent estimates; the experiments ran locally using a quantized model.

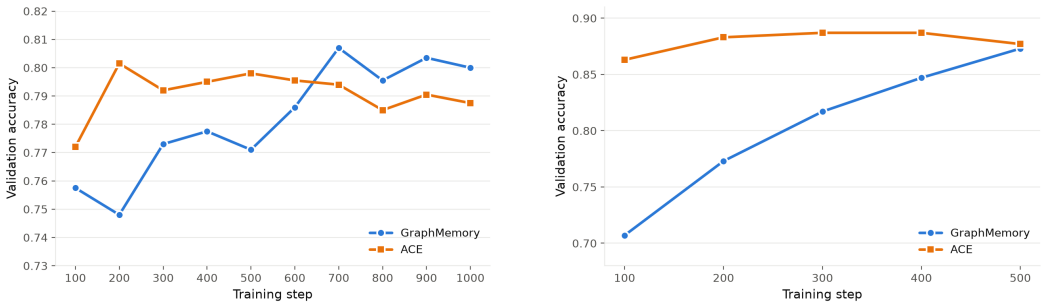


Figure 3: Validation performance of GraphMemory and ACE on Claude across training steps on FINER (left) and FORMULA (right).

token consumption over the full training run and estimate the monetary cost of constructing each memory.

Models We evaluate GraphMemory and ACE using Claude Haiku 4.5 [2] through LiteLLM and a 4-bit quantized Qwen3.8-27B [15] on RTX 6000 Ada GPUs with 48 GB of memory, except on FINER, where topk used an H100 NVL 47-GB MIG allocation and ACE used an H200. For each configuration, we select the memory state that achieves the highest validation accuracy and report its performance on the held-out test set. In the Claude Haiku 4.5 experiments, we cap the ACE memory at 80,000 tokens because its growth would otherwise exceed the model's context-window limit. No such cap is required for Qwen3.8-27B, which supports a larger context window. This highlights the importance of retrieval-based memory: using ACE with smaller context windows may require a memory cap or other context-management techniques.

Datasets Both memory systems are evaluated on the two financial reasoning tasks: FINER [12] and FORMULA [19]. Each system is trained for a single epoch. Our FORMULA split uses 500/300/200 training, validation, and test examples, respectively, while our FINER split uses 1,000/500/441 examples. We evaluate validation performance every 100 training iterations.

Adaptation performance Tables 1 and 2 summarize our findings. Across four dataset–model configurations, the three retrieval methods reduce memory-construction tokens by approximately 83–95%, estimated cost by a similar margin, and test tokens by 79–98%, with accuracy varying by method. On FORMULA with Qwen3.8-27B, LLM retrieval with graph slightly outperforms ACE

Table 2: Active wall-clock time for offline training and held-out test on FORMULA and FINER. All figures are active wall clock. Bold indicates the faster result.

Dataset	Model	System	Training		Final test	
			Total	Per step	Total	Per sample
FORMULA	claude-haiku-4-5	GraphMemory	**6.4 h**	**46 s**	15.7 min	4.7 s
		ACE (80k cap)	8.1 h	58 s	**8.6 min**	**2.6 s**
FINER	claude-haiku-4-5	GraphMemory	**19.0 h**	**68 s**	47.1 min	6.4 s
		ACE (80k cap)	28.9 h	104 s	**26.5 min**	**3.6 s**

using only 16.68% of its training-token budget. With Claude Haiku 4.5, ACE leads this variant by 4.00 percentage points but uses $9.36\times$ as many training tokens. On FINER, this variant trails ACE by 2.94 points with Claude and 4.20 points with Qwen. Topk outperforms embedding retrieval with graph traversal in all four settings. Figure 3 shows that ACE improves rapidly before plateauing or declining as its memory approaches the 80k-token cap. LLM-routed GraphMemory improves more slowly and varies more initially, but reaches its best validation score later on both tasks.

Token and monetary efficiency A large reduction for LLM retrieval with graph occurs on FORMULA with Claude Haiku 4.5, where GraphMemory uses approximately 14M training tokens compared with 131M for ACE, an 89.32% reduction. Its test-time token usage decreases from approximately 18M to 1M, a 92.24% reduction. Comparable savings are observed with Qwen3.8-27B, for which GraphMemory reduces training-token usage from 41M to 7M, and on FINER, where usage falls from 327M to 57M with Claude and from 429M to 31M with Qwen. Under our pricing assumptions, these reductions lower the estimated cost of memory construction from \$157.26 to \$16.80 on Claude/FORMULA, from \$25.44 to \$4.24 on Qwen/FORMULA, from \$392.85 to \$68.23 on Claude/FINER, and from \$267.86 to \$19.24 on Qwen/FINER. Topk has the lowest training-token usage in all four settings.

Wall-clock efficiency As shown in Table 2, GraphMemory also reduces active training time. Training time decreases by 21.0% on FORMULA (6.4 versus 8.1 hours) and 34.3% on FINER (19.0 versus 28.9 hours). GraphMemory is slower during inference, however, requiring approximately $1.8\times$ as much time per test example. This overhead arises because GraphMemory performs retrieval before generating each prediction, whereas ACE directly supplies its complete memory to the backbone.

5 Conclusion, Limitations, and Future Work

We cast agentic memory construction as iterative optimization over textual context, enabling a common analysis of flat and retrieval-based memory. We introduce GRAPHMEMORY, which organizes ACE-style procedural knowledge as a graph and retrieves bounded, query-relevant context. Related memory systems include A-MEM, which links episodic notes in a Zettelkasten-inspired structure [21]; RIZZ, which organizes procedural rules into dynamic branches [7]; and HAGE, which learns graph traversal policies [9]. GraphMemory learns relations between strategies from confidence-weighted helpful and harmful feedback, supporting their composition without requiring predefined edge types or training a retrieval policy. Our formulation shows how bounded retrieval changes memory-context scaling from linear growth under append-only prompting to a constant bound on retrieved content per query. Across all four settings, the three retrieval methods reduce memory-construction tokens by 83–95%, test-time tokens by 79–98%, and estimated API cost similarly. LLM-routed GraphMemory reaches its best validation score later than ACE. Our results remain preliminary, covering two related tasks, two models, and one primary baseline. Broader evaluations, additional memory and prompt-optimization baselines, and repeated runs are needed. Topk outperforms embedding retrieval with graph traversal in all four settings, suggesting limited added utility from the graph; further testing is needed. Finally, longer continual-learning streams will test whether improvements persist beyond one epoch.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026. URL https://arxiv.org/abs/2507.19457.

[2] Anthropic. Claude Haiku 4.5 System Card. Technical report, Anthropic, October 2025. URL https://www.anthropic.com/claude-haiku-4-5-system-card.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. Doc-to-lora: Learning to instantly internalize contexts, 2026. URL https://arxiv.org/abs/2602.15902.

[5] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms. *Advances in Neural Information Processing Systems*, 37:71596–71642, 2024.

[6] Chen Dun, Mirian Hipolito Garcia, Guoqing Zheng, Ahmed Hassan Awadallah, Anastasios Kyrillidis, and Robert Sim. Sweeping heterogeneity with smart mops: Mixture of prompts for llm task adaptation, 2025. URL https://arxiv.org/abs/2310.02842.

[7] Sonali Goel, Pranav Vaidhyanathan, Lucas Schorling, Natalia Ares, and Maike Osborne. Rizz: Routing interactions to near zero-interference zones for continual adaptation of black-box agents. *arXiv preprint arXiv:2606.20638*, 2026.

[8] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[9] Dongming Jiang, Yi Li, Guanpeng Li, Qiannan Li, and Bingzhe Li. Hage: Harnessing agentic memory via rl-driven weighted graph evolution. *arXiv preprint arXiv:2605.09942*, 2026.

[10] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL https://arxiv.org/abs/2310.03714.

[11] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)*, pages 4582–4597, 2021.

[12] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. Finer: Financial numeric entity recognition for xbrl tagging. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4419–4431, 2022.

[13] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.

[14] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs, 2024. URL https://arxiv.org/abs/2406.11695.

[15] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026. URL https://qwen.ai/blog?id=qwen3.8.

[16] Keshav Ramji, Tahira Naseem, and Ramón Astudillo. Latent principle discovery for language model self-improvement. *Advances in Neural Information Processing Systems*, 38:162283–162336, 2026.

[17] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory, 2025. URL https://arxiv.org/abs/2504.07952.

[18] Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, and Tu Vu. Wikiskill: Compiling agent experience into persistent knowledge for skill evolution. *arXiv preprint arXiv:2608.27454*, 2026.

[19] Dannong Wang, Jaisal Patel, Daochen Zha, Steve Y Yang, and Xiao-Yang Liu. Finlora: Benchmarking lora methods for fine-tuning llms on financial datasets. *arXiv preprint arXiv:2505.19819*, 2025.

[20] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China information sciences*, 68(2):121101, 2025.

[21] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38: 17577–17604, 2026.

[22] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024.

[23] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers, 2024. URL https://arxiv.org/abs/2309.03409.

[24] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic" differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

[25] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2026. URL https://arxiv.org/abs/2510.04618.

A Additional Formalization

A.1 Textual Optimization Oracles

Equation (2) suppresses how the candidate pool is constructed. MIPROv2 searches over candidate instructions and demonstrations for each program module. If $\mathcal{I}_j$ and $\mathcal{E}_j$ denote the instruction and demonstration candidates for module j, respectively, its search space can be abstracted as

$$\mathcal{Z} = \prod_{j=1}^{m} \big(\mathcal{I}_j \times \mathcal{E}_j\big), \qquad \widehat{z}_T \in \arg\min_{z \in \mathcal{Z}_T} L_{\mathcal{D}}\big(c(z)\big), \tag{9}$$

where $\mathcal{Z}_T \subseteq \mathcal{Z}$ contains the configurations evaluated within budget T, and $c(z)$ denotes their induced context [14].

GEPA constructs its candidate set adaptively rather than drawing it from a fully specified product space. Starting from an archive $\mathcal{A}_0$, it uses execution traces and feedback to reflect on selected candidates and add new proposals:

$$\mathcal{A}_{t+1} = \mathcal{A}_t \cup \mathrm{Propose}(c_t, \tau_t), \qquad \widehat{c}_T \in \arg\min_{c \in \mathcal{A}_T} L_{\mathcal{D}}(c). \tag{10}$$

Here $\mathrm{Propose}$ abstracts GEPA's reflective mutation and candidate combination operations [1]. Thus, $\mathcal{P}_T$ in Equation (2) may be either a searched subset of a larger predefined space, as in MIPROv2, or an archive grown during optimization, as in GEPA. In both cases, we abstract the resulting optimized program or prompt as a context c that is fixed during test-time inference.

A second family uses Equation (3) to propose textual changes iteratively. OPRO conditions the optimizer on previously proposed contexts and their scores. TextGrad generates textual feedback and propagates it through the computation that produced the output. TRACE uses output feedback together with execution traces to localize the part of the context to be modified [22, 24, 5]. When restricted to context optimization, these methods can be viewed as different realizations of $\mathcal{O}$. The gradient-descent analogy concerns the shared update structure, not differentiability or a convergence guarantee. In particular, Δc_t is only a proposed direction unless an evaluation or acceptance step verifies that it reduces the objective.

A.2 Why Use a Local Context-Block Surrogate?

Allowing a separate Δ_i for each example enlarges the search space and can produce a maximally biased solution: an unconstrained block could simply encode the answer to its source input. Equation (4) is therefore not a reformulation with the same minimizer as Equation (1). It is a surrogate for constructing the elements of a reusable memory.

The motivation is the asymmetry between classical optimization and textual optimization. If a reliable descent direction for the global context were available, decomposing the problem into per-example variables would offer little benefit and would create an obvious generalization problem. In the black-box setting, however, the update direction must itself be produced by an imperfect model-based oracle. Asking that oracle to extract one short lesson from one observed success or failure is a more localized generation task than asking it to rewrite a growing global context to improve average performance over all examples.

Generalization is delegated to the memory interface. Given a memory bank δ, a composer or retriever constructs $c_\phi(x) = \phi(\delta, x)$ for a new input. A block is useful only when it encodes reusable information and ϕ exposes it on inputs for which that information is relevant. This is why context-block quality cannot be judged only by Equation (4); final validation and test risk must still be measured using the composed or retrieved contexts, as in Equation (16) for GraphMemory.

A.3 Memory-Context Scaling

Let $L_0 = \mathrm{len}(c_\delta^{(0)})$ and $b_i = \mathrm{len}(\Delta_i)$. Under the append-only update in Equation (5),

$$\begin{aligned} \mathrm{len}\left(c_\delta^{(i)}\right) &= \mathrm{len}\left(c_\delta^{(i-1)}\right) + b_i, \\ \mathrm{len}\left(c_\delta^{(N)}\right) &= L_0 + \sum_{i=1}^{N} b_i \le L_0 + NB = O(N). \end{aligned} \tag{11}$$

If the blocks have positive average length bounded away from zero, the final memory-context length is $\Theta(N)$.

The cumulative number of memory-context tokens exposed across the N updates obeys

$$\begin{aligned}\sum_{i=1}^{N} \operatorname{len}\left(c_\delta^{(i)}\right) &= NL_0 + \sum_{j=1}^{N}(N-j+1)b_j \\ &\le NL_0 + \frac{BN(N+1)}{2} = O(N^2).\end{aligned} \tag{12}$$

This is a bound on memory tokens supplied to the model, not on all prompt and completion tokens used by the complete system.

For retrieval-based memory, each context is constructed afresh and does not contain the context retrieved for the preceding example. If $\operatorname{len}(\Delta_i) \le B$ and at most K blocks are retrieved, then for every i,

$$\operatorname{len}\left(c_\phi^{(i)}(x_i)\right) \le KB = O(1), \qquad \sum_{i=1}^{N} \operatorname{len}\left(c_\phi^{(i)}(x_i)\right) \le NKB = O(N). \tag{13}$$

Consequently, the per-query memory context is $O(1)$. Across N examples, the cumulative memory-context exposure is $O(N)$. These bounds exclude the stored graph, condensed index, and any context consumed internally by ϕ.

A.4 GraphMemory as Structured Retrieval

At step t, GraphMemory stores the blocks in $G^{(t)} = (V^{(t)}, E^{(t)})$. A new node $v^{(t)}$ stores $\Delta^{(t)}$, while $R^{(t)}$ contains the relations introduced or updated for that node. Under the same append-only simplification as Equation (5),

$$V^{(t+1)} = V^{(t)} \cup \{v^{(t)}\}, \qquad E^{(t+1)} = E^{(t)} \cup R^{(t)}. \tag{14}$$

The implementation may additionally edit or merge nodes and revise edge scores; those operations do not alter the retrieval interface below.

For an input x, the retriever selects a query-dependent subset of nodes and serializes their blocks:

$$\begin{aligned}S^{(t)}(x) &= \phi\big(G^{(t)}, x\big) \subseteq V^{(t)}, \\ c_G^{(t)}(x) &= \bigoplus_{v_j \in S^{(t)}(x)} \Delta_j.\end{aligned} \tag{15}$$

For fixed ϕ, the empirical risk of the graph-induced contexts is

$$L_{\mathcal{D}}^{\text{graph}}\big(G^{(t)}\big) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\left[\ell\big(f(c_G^{(t)}(x_i), x_i), y_i\big)\right]. \tag{16}$$

GraphMemory constructs $G^{(0)}, G^{(1)}, \ldots$ with the aim of reducing this risk. At the level of memory exposure, supplying the full memory is recovered as the special case $S^{(t)}(x) = V^{(t)}$ for every x. GraphMemory instead uses a bounded, query-dependent set, so a specialized block affects only the inputs for which it is retrieved.

B Implementation Details

B.1 Directed Evidence-Weighted Memory Graph

GraphMemory maintains a directed graph $G^{(t)} = (V^{(t)}, E^{(t)})$. Each node stores a unique identifier, a short routing key, and the full ACE-style bullet point used as context. The routing index contains only the identifier and short key; full bullet-point content is loaded only after graph retrieval. A directed edge $e = (u, v)$ records that the knowledge in node v may be useful after node u has been retrieved. The implementation does not store a single mutable weight. Instead, each directed edge

stores two nonnegative, confidence-weighted evidence counters: $S_e^{(t)}$ for helpful transitions and $F_e^{(t)}$ for harmful transitions. If edge e is created at step τ_e, its counters are initialized as

$$S_e^{(\tau_e)} = F_e^{(\tau_e)} = 0. \tag{17}$$

We treat these counters as fractional pseudo-counts under a uniform $\mathrm{Beta}(1,1)$ prior. Under this fractional-observation interpretation, the edge parameter is

$$\theta_e \mid \mathcal{H}_t \sim \mathrm{Beta}\Big(1 + S_e^{(t)},\, 1 + F_e^{(t)}\Big), \tag{18}$$

where $\mathcal{H}_t$ is the evidence accumulated through step t. The value used to rank the edge during retrieval is the posterior mean

$$w_e^{(t)} = \mathbb{E}[\theta_e \mid \mathcal{H}_t] = \frac{S_e^{(t)} + 1}{S_e^{(t)} + F_e^{(t)} + 2}. \tag{19}$$

Thus, a new edge has neutral weight $w_e^{(\tau_e)} = 1/2$ even though no explicit weight field is stored. Because the graph is directed, (u, v) and (v, u) have separate counters and can receive different retrieval weights.

After generation, the Generator reports an ordered influence trace $M_t = [n_1, \ldots, n_k]$ containing the loaded memory nodes that it attributes as influential to its answer. Only identifiers among the nodes loaded for that example are accepted. Adjacent items define candidate directed transitions,

$$\mathcal{T}(M_t) = \{(n_1, n_2), (n_2, n_3), \ldots, (n_{k-1}, n_k)\}. \tag{20}$$

The Reflector labels each candidate as helpful, harmful, or neutral and returns an evidence strength $c_e^{(t)} \in [0, 1]$. For an existing edge e, the evidence update is

$$\Big(S_e^{(t+1)}, F_e^{(t+1)}\Big) = \begin{cases} \Big(S_e^{(t)} + c_e^{(t)}, F_e^{(t)}\Big), & \text{if } e \text{ is helpful,} \\ \Big(S_e^{(t)}, F_e^{(t)} + c_e^{(t)}\Big), & \text{if } e \text{ is harmful,} \\ \Big(S_e^{(t)}, F_e^{(t)}\Big), & \text{if } e \text{ is neutral.} \end{cases} \tag{21}$$

The confidence value therefore controls the fractional amount of evidence contributed by one observation; neutral feedback and $c_e^{(t)} = 0$ leave both counters unchanged, as do edges that were not observed on the example. Hallucinated or unloaded node identifiers, duplicate pairs, invalid labels, and confidences outside $[0, 1]$ are discarded before this update, so each unique transition is updated at most once per example. Observing a transition does not implicitly create an edge: new edges can be introduced only by validated Curator operations, with both counters initialized to zero. The Curator may likewise issue a separate validated edge-removal operation when a relation should no longer be retained; deletion is not an automatic consequence of Equation (19). Node additions and revisions are also local, and new nodes are inserted into the routing index without rewriting the remainder of the graph.

The retrieval weight is a smoothed estimate of how often the Reflector has judged an observed directed transition helpful under the current retrieval policy. It should not be interpreted as a causal measure of interaction between two nodes or as a calibrated probability that retrieving the edge will improve accuracy. Evidence is policy-dependent because highly ranked edges are more likely to be observed again, and ranking by the posterior mean does not explicitly reward uncertainty. These limitations are important when interpreting the learned graph.

B.2 Retrieval and Traversal

Retrieval has two stages. First, an LLM Retriever reads the compact index and returns a structured selection of up to two index sections and at most six entry nodes. Second, GraphMemory performs a weight-prioritized breadth-first traversal over outgoing edges. Entry nodes have depth zero. At each breadth level, active outgoing neighbors are ordered by decreasing posterior-mean weight from Equation (19), with target-node identifier used as a deterministic tie-break, and at most the two highest-scoring neighbors of a loaded node are considered. Traversal is bounded by two hops from each entry node, at most 12 loaded nodes, and at most 2,500 tokens of serialized node content. A node is not loaded if doing so would violate either the node or token budget.

Algorithm 1 Bounded posterior-mean-prioritized retrieval in GraphMemory

Require: Query x, directed graph $G = (V, E)$, compact index $I(G)$
$A \leftarrow \mathrm{LLMSelect}(I(G), x)$ {at most 2 sections and 6 valid entry nodes}
$Q \leftarrow [(v, 0) : v \in A]$, $L \leftarrow [\,]$, $Z \leftarrow \varnothing$
while $Q \neq \varnothing$ and $|L| < 12$ **do**
  $(v, d) \leftarrow \mathrm{popFront}(Q)$
  if $v \notin Z$ **then**
    $Z \leftarrow Z \cup \{v\}$
    if $\mathrm{len}(\mathrm{Ser}(L \parallel [v])) \leq 2500$ **then**
      append v to L
      if $d < 2$ **then**
        $N_v \leftarrow \mathrm{Top2}_{u:(v,u)\in E}\, w_{(v,u)}$
        append $[(u, d + 1) : u \in N_v]$ to Q
      end if
    end if
  end if
end while
return $\mathrm{Ser}(L)$

Algorithm 1 gives the retrieval procedure. The queue preserves breadth-first depth order, while edge scores determine the priority among neighbors at the same expansion point. Duplicate nodes reached from different entries are loaded only once.

The Retriever uses structured JSON output. Invalid identifiers are discarded. We use no full-memory fallback: if the graph is empty or no valid entry node is selected, the retrieved memory is empty and generation proceeds without a memory block. This choice prevents retrieval failures from silently reverting to the full-playbook exposure that GraphMemory is designed to avoid.

For each retrieval, the implementation records the selected section and entry identifiers, loaded node identifiers, considered and traversed edges, loaded memory tokens, maximum hop depth, traversal time, and whether a fallback was used. These traces both support token and latency accounting and restrict relation updates to content that was actually available during generation.

B.3 Retrieve–Generate–Reflect–Curate Execution Order

For training example (x_t, y_t), GraphMemory first retrieves from the graph constructed from earlier examples,

$$c_t = \phi(G^{(t)}, x_t), \qquad \widehat{y}_t = f(c_t, x_t). \tag{22}$$

Only after producing $\widehat{y}_t$ does the Reflector inspect the prediction, available feedback, and execution trace. The Curator then emits local graph edits, which are applied to obtain $G^{(t+1)}$. Consequently, information derived from y_t cannot affect the prediction on x_t; it becomes available only to subsequent examples. The Generator and Reflector retain ACE's core roles, with the additional memory-use tracing and edge feedback described above, while the Retriever and graph-valued Curator implement the two GraphMemory-specific interfaces described in Section 3.

B.4 Model and Evaluation Configuration

For the Qwen3.8-27B experiments, the model is loaded through Hugging Face. We retain the checkpoint configuration except for three explicit runtime choices: 4-bit quantization, scaled dot-product attention (SDPA), and disabled thinking mode. Disabling thinking prevents unnecessarily long reasoning outputs. These model and inference settings are identical for ACE and GraphMemory. For the Claude Haiku 4.5 experiments, all model calls are made through LiteLLM using its default request configuration. Within each dataset–backbone comparison, ACE and GraphMemory use the same backbone and model configuration; the method-specific difference is the memory interface rather than the underlying language model.

Each system processes the training split for one epoch, and the then-current memory is evaluated on the validation split every 100 updates. The checkpoint with the highest validation accuracy is selected

for the held-out test evaluation. In the Claude experiments, ACE uses the 80,000-token memory cap reported in the main text. Formula uses 500/300/200 train/validation/test examples, and FiNER uses 1,000/500/441 examples.

B.5 Token and Latency Accounting

The bounded-retrieval result in Proposition 2 applies to the serialized node content supplied to the Generator. It does not include the compact index read by the Retriever. Because the present Retriever reads the complete index, its input grows with the number of indexed nodes. Total system usage includes the Retriever's index, retrieved node content, fixed instructions and query tokens, and the remaining input and output tokens from the Retriever, Generator, Reflector, and Curator. Only the retrieved node content supplied to the Generator is covered by the constant retrieved-content bound. The wall-clock measurements include the additional retrieval call made by GraphMemory, which explains why GraphMemory can use fewer tokens while having higher per-query latency.